

Anthropic Prompt Caching Study Guide

Created: 2026-02-01

Goal: Master prompt caching to reduce costs by 90% and improve latency

Time: 30-45 minutes

■ What You'll Learn

1. What prompt caching is and why it matters
 2. How caching works (TTL, cache breakpoints, pricing)
 3. Practical implementation patterns
 4. Real-world use cases for your projects
 5. Gotchas and best practices
-

■ Part 1: Core Concepts (10 min)

What is Prompt Caching?

Anthropic caches parts of your prompt to avoid reprocessing the same content repeatedly.

Key insight: Large, static context (docs, examples, system prompts) gets expensive when sent on every API call. Caching makes repeated use nearly free.

The Numbers

Without caching: - Input tokens: \$3.00 per million tokens (Sonnet 3.5) - Output tokens: \$15.00 per million tokens

With caching: - Cache write: \$3.75 per million tokens (25% premium, one-time) - Cache read: \$0.30 per million tokens (**90% discount!**) - Output tokens: \$15.00 (unchanged)

Example calculation: - 100K token system prompt - 10 API calls - Without cache: $100K \times 10 \times \$3.00/1M = \3.00 - With cache: $(100K \times \$3.75/1M) + (100K \times 9 \times \$0.30/1M) = \$0.375 + \$0.27 = \$0.645$ (78% savings)

How It Works

1. **Mark cacheable content** with `cache_control: {type: "ephemeral"}`
2. **First request:** Anthropic processes and caches marked content (cache write)
3. **Subsequent requests:** Cached content is reused (cache read - 90% off!)
4. **Expiration:** Cache lasts **5 minutes** from last use

■ Part 2: Technical Details (15 min)

Cache Breakpoints

You can mark **up to 4 cache breakpoints** per request:

```
{
  "model": "claude-sonnet-4-5",
  "max_tokens": 1024,
  "system": [
    {
      "type": "text",
      "text": "You are a helpful assistant...",
      "cache_control": {"type": "ephemeral"} // ← Cache breakpoint 1
    }
  ],
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
```

```

    "text": "Here's a large document: ...",
    "cache_control": {"type": "ephemeral"} // ← Cache breakpoint 2
  },
  {
    "type": "text",
    "text": "What does it say about X?" // ← Not cached
  }
]
}
]
}

```

Rules: - Cache control applies to **that block and everything before it** - Place breakpoints at natural boundaries (system prompt, docs, examples) - Last ~1024 tokens before breakpoint must stay identical for cache hit

TTL (Time To Live)

- **Duration:** 5 minutes from last access
- **Auto-extension:** Each cache hit resets the 5-minute timer
- **Expiration:** After 5 min of inactivity, cache is cleared

Implication: Keep requests flowing within 5 min for maximum savings.

Cache Warming

First request in a session: - Marked content gets cached (cache write cost) - Subsequent requests reuse the cache (cache read cost)

Strategy: If you know you'll make multiple calls, the first call "warms" the cache.

■ Part 3: Use Cases & Patterns (10 min)

Use Case 1: Large System Prompts

Scenario: Your agent has a 50K token system prompt with tools, examples, personality.

Without caching: - Every call: $50K \text{ input tokens} \times \$3/1M = \$0.15$

With caching: - First call: $50K \times \$3.75/1M = \0.1875 - Next 9 calls: $50K \times 9 \times \$0.30/1M = \0.135 - Total for 10 calls: $\$0.3225$ (vs $\$1.50$ without cache) = **78% savings**

Implementation:

```
"system": [  
  {  
    "type": "text",  
    "text": "<entire-50k-system-prompt>",  
    "cache_control": {"type": "ephemeral"}  
  }  
]
```

Use Case 2: RAG with Long Documents

Scenario: RAG system retrieves 100K tokens of context per query.

Pattern:

```
"messages": [  
  {  
    "role": "user",  
    "content": [  
      {  
        "type": "text",  
        "text": "Context:\n<retrieved-docs>",  
        "cache_control": {"type": "ephemeral"} // Cache the docs  
      },  
      {  
        "type": "text",  
        "text": "Question: What is X?" // Don't cache the question  
      }  
    ]  
  }  
]
```

Key: If multiple users ask about the same document, they all benefit from cache.

Use Case 3: Multi-Turn Conversations

Scenario: Chat interface with growing conversation history.

Strategy: - Cache the stable history (older messages) - Don't cache the latest turn

Example (turn 5):

```
"messages": [  
  {"role": "user", "content": "Turn 1 question"},  
  {"role": "assistant", "content": "Turn 1 answer"},  
  {"role": "user", "content": "Turn 2 question"},  
  {"role": "assistant", "content": "Turn 2 answer"},  
  {  
    "role": "user",  
    "content": [  
      {  
        "type": "text",  
        "text": "Turn 3 question",  
        "cache_control": {"type": "ephemeral"} // Cache up to here  
      }  
    ]  
  },  
  {"role": "assistant", "content": "Turn 3 answer"},  
  {"role": "user", "content": "Turn 4 question"}, // Fresh content  
  {"role": "assistant", "content": "Turn 4 answer"},  
  {"role": "user", "content": "Turn 5 question"} // Fresh content  
]
```

Note: This is tricky - cache keys change as conversation grows. Best for recent history that's stable.

Use Case 4: Code Analysis Tools

Scenario: Code review agent analyzing a large codebase.

Pattern: - Cache the entire codebase context - Vary the analysis questions

```
"messages": [  
  {  
    "role": "user",  
    "content": [  
      {  
        "type": "text",  
        "text": "Codebase:\n<paste-entire-repo>",
```

```
    "cache_control": {"type": "ephemeral"}
  },
  {
    "type": "text",
    "text": "Find security vulnerabilities"
  }
]
}
```

Benefit: Multiple analysis passes over same code = 90% off after first pass.

■ ■ Part 4: Gotchas & Best Practices (10 min)

Gotchas

1. **Minimum cache size: ~1024 tokens**
2. Don't cache tiny snippets (cache overhead > savings)

Anthropic recommends 2048+ tokens for meaningful benefit

Cache key = exact content

5. Single character change = cache miss
6. Whitespace matters!

Order matters!

TTL resets per hit, not per request

9. If cache isn't hit, TTL keeps ticking

Plan request timing

Cache warming cost

12. First request pays 25% premium

Only worth it if you'll make 2+ requests

Not all models support caching

15. Check model compatibility (Claude 3.5 Sonnet/Opus, Claude 3 Haiku/Opus)

Best Practices

- **Cache static, large content** - System prompts - Documentation - Few-shot examples - Retrieved context (RAG)
 - **Don't cache dynamic content** - User queries - Timestamps - Session-specific data
 - **Place breakpoints strategically** - After system prompt - After large document dumps - After stable conversation history
 - **Monitor cache hit rates** - Track `usage.cache_creation_input_tokens` (writes) - Track `usage.cache_read_input_tokens` (hits) - Calculate hit rate: $\text{cache_read} / (\text{cache_read} + \text{cache_creation})$
 - **Batch requests when possible** - Keep requests within 5-min window - Amortize cache write cost
 - **Version your cached content** - If system prompt changes, cache invalidates - Consider versioning strategy for updates
-

■ ■ Part 5: Your Implementation Plan (5 min)

For Your MCP Dev Crew (aidev)

Current setup: - Claude Code + Gemini CLI agents - System prompts with tool definitions - Likely 10K-50K tokens per agent

Caching strategy: 1. Add cache control to system prompts 2. Measure current input token usage 3. Calculate expected savings 4. Monitor cache hit rates

Expected savings: 60-80% on input tokens (after cache warming)

Quick Win: Cache System Prompt

Before:

```
response = client.messages.create(  
    model="claude-sonnet-4-5",  
    max_tokens=1024,  
    system="<your-system-prompt>",  
    messages=[...]  
)
```

After:

```
response = client.messages.create(  
    model="claude-sonnet-4-5",  
    max_tokens=1024,  
    system=[  
        {  
            "type": "text",  
            "text": "<your-system-prompt>",  
            "cache_control": {"type": "ephemeral"}  
        }  
    ],  
    messages=[...]  
)
```

That's it! First call caches, subsequent calls reuse.

■ Success Metrics

Track these to measure impact:

1. **Cost reduction**
2. Before: Total input token cost
3. After: Cache write + cache read cost

Target: 60-80% reduction

Cache hit rate

6. $\text{cache_read} / (\text{cache_read} + \text{cache_creation})$

Target: >80% (after cache warming)

Latency improvement

9. Cache reads are faster (less processing)

Expected: 10-30% faster responses

Cache efficiency

12. Are you keeping requests within 5-min window?
13. Are cache misses due to content changes or timing?

■ Next Steps

After this study guide:

1. ■ Read official docs: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>
2. ■ Identify your largest static prompts (aidev system prompts)
3. ■ Implement caching on one agent
4. ■ Measure savings
5. ■ Roll out to all agents
6. ■ Create ADR documenting the decision

Estimated time to implement: 30 minutes

Estimated savings: \$50-200/month (depends on usage)

■ Resources

- **Official docs:** <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>
 - **Pricing:** <https://www.anthropic.com/pricing>
 - **API reference:** <https://docs.anthropic.com/en/api/messages>
 - **Your aidev crew:** `\aidev\`
-

Study time: ~45 minutes

Implementation time: ~30 minutes

ROI: 60-80% cost reduction on input tokens ■

Ready to implement? Start with Task #3 in Molt: "■ Implement Prompt Caching in aidev Crew"!